

INFORME TÉCNICO DE AUDITORÍA DE SEGURIDAD INTEGRAL

**Análisis Avanzado de Calidad de Software,
Vulnerabilidades Lógicas y Control de Deuda Técnica**

Evaluación Estática y Dinámica de la Plataforma CarteraPro Risk Lab

Firma Consultora / Grupo Auditor:

Xiomara Ocampo Hurtado

Carlos Augusto Aranzazu

María Luisa Londoño

Asignatura:

Tendencias en Ingeniería de Software

Docente Encargado:

Carlos Andrés Jaramillo

Universidad Alexander von Humboldt
Facultad de Ingeniería y Ciencias Básicas
Programa de Ingeniería de Software
Armenia, Quindío

2026

Índice

1. Introducción y Alcance	6
1.1. Objetivo de la Auditoría	6
1.2. Contexto del Compromiso	6
1.3. Alcance del Análisis Técnico	6
1.4. Resumen Cuantitativo de Indicadores y Severidad	7
2. Metodología de la Auditoría	7
3. Arquitectura Identificada y Funcionamiento de Capas	8
3.1. Capa del Cliente (React Presentation Layer)	8
3.2. Capa de Servicio y Base de Datos Relacional	8
4. Hallazgos de Calidad de Software y Deuda Técnica	9
4.1. CAL-001: Duplicación de Código Exacta en Cálculo de Mora	9
4.2. CAL-002: Complejidad Ciclomática Excesiva en Evaluación de Riesgo	9
4.3. CAL-003: Manejo Desafortunado de Errores (Swallowed Exceptions)	10
4.4. CAL-004: Componentes Monolíticos Complejos (Funciones Diosas)	10
4.5. CAL-005: Código Muerto y Variables Locales Obsoletas	10
4.6. CAL-006: Uso Sistémico e Inadecuado del Operador var	11
4.7. CAL-007: Ausencia Total de Documentación y Comentarios Técnicos	11
4.8. CAL-008: Validación Mediante Cadenas de Texto Rígidas (Hardcoded Strings)	11
4.9. CAL-009: Complejidad Cognitiva Elevada en Event Loop	12
4.10. CAL-010: Inexistencia Absoluta de Suites de Pruebas Automatizadas	12

5. Hallazgos de Seguridad (OWASP Top 10)	13
5.1. SEC-001: Inyección SQL Estricta en Módulo de Autenticación	13
5.2. SEC-002: Ejecución Remota de Comandos (OS Command Injection) . .	13
5.3. SEC-003: Control de Acceso Roto mediante Token Predictible Artesanal	14
5.4. SEC-004: Fuga Masiva de Credenciales en Texto Plano	14
5.5. SEC-005: Exposición de Variables de Entorno del Servidor	14
5.6. SEC-006: Configuración Permisiva de Cabeceras CORS	15
5.7. SEC-007: Ausencia del Atributo HttpOnly en la Emisión de Cookies . .	15
5.8. SEC-008: Llave Criptográfica JWT Semilla Quemada en Código	15
5.9. SEC-009: Inyección de Salto de Directorios (Path Traversal)	16
5.10. SEC-010: Vulnerabilidad de Cross-Site Scripting Almacenado	16
5.11. SEC-011: Ausencia de Mecanismos de Control de Tasa (Rate Limiting)	16
5.12. SEC-012: Puerta Trasera de Auto-login por Parámetro URL	17
5.13. SEC-013: Redirecciones Abiertas Inseguras	17
5.14. SEC-014: Ausencia de Cabeceras de Seguridad HSTS	17
5.15. SEC-015: Fuga de Stack Traces ante Errores Internos HTTP 500	17
5.16. SEC-016: Manejo de Sesiones Infinitas sin Expiración Controlada . . .	18
5.17. SEC-017: Falta de Validación de Esquemas y Tipos en Datos de Entrada	18
5.18. SEC-018: Exposición de Secretos de Desarrollo en el Bundle de Producción	18
5.19. SEC-019: Ausencia Total de Cabeceras Content-Security-Policy (CSP) .	19
5.20. SEC-020: Implementación de Algoritmia Hashing MD5 Obsoleta	19
5.21. SEC-021: Divulgación Tecnológica Explícita en Cabecera X-Powered-By	19
5.22. SEC-022: Falta de Cabeceras Contra Suplantación de Tipos MIME . .	20
5.23. SEC-023: Vulnerabilidad ante Secuestro de Clics (Clickjacking)	20
5.24. SEC-024: Inyección de Encabezados HTTP (HTTP Response Splitting)	20

5.25. SEC-025: Almacenamiento Inseguro de Sesiones en LocalStorage	21
5.26. SEC-026: Falta de Sanitización Estricta en la Subida de Archivos . . .	21
5.27. SEC-027: Endpoint de Depuración Reset Público Activo en Producción	21
5.28. SEC-028: Claves de Cifrado Simétrico Estáticas Compartidas	22
6. Análisis de Composición de Software (SCA)	23
6.1. DEP-001: Biblioteca request Deprecada y Vulnerable (CVE-2023-28155)	23
6.2. DEP-002: Paquete Transitivo form-data Inseguro por Semillas Previsibles	23
6.3. DEP-003: Biblioteca Axios Vulnerable a la Fuga de Tokens CSRF . . .	23
6.4. DEP-004: Express Framework Vulnerable a Ejecuciones XSS en Redirección	24
6.5. DEP-005: JsonWebToken Obsoleto con Omisión de Firma (Algoritmo None)	24
6.6. DEP-006: Lodash Desactualizado Propenso a Prototype Pollution . . .	24
6.7. DEP-007: Moment.js Legacy con Fallos de Expresiones Regulares (ReDoS)	24
6.8. DEP-008: Multer Vulnerable a Agotamiento de Recursos RAM	25
6.9. DEP-009: Body-Parser Bloqueante por Cargas Masivas de Datos	25
6.10. DEP-010: Driver SQLite3 Bindings Vulnerable a Fallos de Segmentación	25
6.11. DEP-011: Servidor de Desarrollo Vite Vulnerable a Path Traversal . . .	26
6.12. DEP-012: Nodemon Inseguro con Inyecciones en Scripts de Consola . .	26
6.13. DEP-013: Dotenv Obsoleto ante Caracteres de Escape Inválidos	26
6.14. DEP-014: Cors Package Legacy con Asignación de Orígenes Laxos . . .	26
6.15. DEP-015: Multer File Type Spoofing en Detección de Extensiones . . .	27
7. Valoración de Riesgos e Impacto de Negocio	28
8. Correlación y Validación Cruzada de Hallazgos	29

9. Anexo de Evidencias Técnicas	31
9.1. Evidencia 1: Reproducción Manual de Bypass de Autenticación (SEC-001)	31
9.2. Evidencia 2: Exfiltración de Tabla de Usuarios vía SQLi UNION (SEC-002)	31
9.3. Evidencia 3: Ejecución Remota de Comandos (RCE via exec - SEC-004)	32
9.4. Evidencia 4: Exposición de Secretos en Local File Inclusion (SEC-013)	32
9.5. Evidencia 5: Activación de la Backdoor de Auto-login por URL (SEC-022)	33
 10. Conclusiones y Plan de Intervención Priorizado	 34
10.1. Horizonte 1: Acciones de Emergencia Inmediatas (1 a 2 semanas)	34
10.2. Horizonte 2: Mitigaciones y Parches a Corto Plazo (1 mes)	34
10.3. Horizonte 3: Sostenibilidad e Ingeniería a Mediano Plazo (3 meses)	35
10.4. Recomendaciones Complementarias de Ingeniería	35
10.5. Declaración Final del Equipo Auditor	36

Índice de tablas

1.	Componentes Tecnológicos Incluidos en el Alcance	6
2.	Distribución de Hallazgos por Categoría y Severidad	7
3.	Matriz Multidimensional de Riesgos Financieros y Técnicos	28
4.	Correlación Híbrida de Detección de Infracciones Técnicas	30

Índice de figuras

1. Introducción y Alcance

1.1. Objetivo de la Auditoría

El presente documento constituye el entregable técnico final del proceso de auditoría técnica independiente bajo la modalidad de caja blanca realizado sobre la plataforma web denominada CarteraPro Risk Lab. El propósito fundamental de este análisis es identificar, clasificar, valorar y proponer estrategias de remediación aplicables a las deficiencias lógicas de diseño, acumulación de deuda técnica, vulnerabilidades dinámicas y estáticas, y riesgos de infraestructura que comprometan las operaciones de la aplicación.

1.2. Contexto del Compromiso

La aplicación auditada gestiona flujos informativos financieros de alta sensibilidad corporativa, incluyendo bases de datos de deudores, balances de facturación y módulos de parametrización de riesgos de crédito. El análisis fue realizado íntegramente en un entorno de laboratorio controlado y aislado (localhost), con acceso total al código fuente, archivos de configuración e instrucciones de despliegue.

1.3. Alcance del Análisis Técnico

El espectro analítico abarcó el cien por ciento de las capas del repositorio de software:

Tabla 1: Componentes Tecnológicos Incluidos en el Alcance

Componente	Tecnología Base	Incluido en Alcance
Backend (Servidor API)	Node.js + Express 4.17.1	Sí
Frontend (Aplicación Web)	React 18.2 + Vite 5.0	Sí
Base de Datos	SQLite (Archivo local)	Sí
Configuración de Entorno	Docker, .env, docker-compose	Sí
Dependencias de Terceros	npm (Backend y Frontend)	Sí
Infraestructura de Red	Red local de laboratorio	Parcial
Entorno de Producción	No disponible	No

1.4. Resumen Cuantitativo de Indicadores y Severidad

A través del uso de herramientas automatizadas (SonarQube, OWASP ZAP, npm audit) coordinadas con inspección manual línea por línea, se indexaron un total de 38 hallazgos principales en el sistema:

Tabla 2: Distribución de Hallazgos por Categoría y Severidad

Severidad	Calidad de Software	Seguridad (OWASP)	Total
Crítica	0	14	14
Alta	5	8	13
Media	4	6	10
Baja	1	0	1
Total	10	28	38

2. Metodología de la Auditoría

La investigación técnica aplicó un marco analítico híbrido compuesto por cuatro metodologías complementarias de aseguramiento:

- **SAST (Static Application Security Testing):** Escaneo e inspección estática del árbol de código fuente invocando la plataforma local SonarQube Community Edition, parametrizada bajo las reglas de desarrollo limpio para entornos ECMAScript.
- **DAST (Dynamic Application Security Testing):** Inyección activa automatizada y pasiva controlada en runtime de solicitudes HTTP anómalas utilizando el proxy interceptor OWASP ZAP versión 2.15 sobre los puertos activos del servidor de pruebas.
- **SCA (Software Composition Analysis):** Extracción e inspección automatizada de la paquetería de terceros declarada en los manifiestos del proyecto a través de la herramienta npm audit, cruzando los resultados con las bases de datos públicas de vulnerabilidades conocidas (CVE).
- **Ingeniería Inversa y Code Review Manual:** Inspección minuciosa línea por línea de los controladores de rutas, lógica de enrutamiento y funciones encargadas

del almacenamiento de sesiones para detectar fallas que evaden herramientas automáticas.

3. Arquitectura Identificada y Funcionamiento de Capas

El sistema opera bajo una arquitectura cliente-servidor desacoplada:

3.1. Capa del Cliente (React Presentation Layer)

El cliente procesa las mutaciones de estados e interactúa con el usuario a través del árbol del DOM de React. Las llamadas hacia servicios protegidos se despachan implementando el cliente HTTP Axios, canalizando el flujo de tráfico hacia el proxy de desarrollo local de Vite en el puerto 5173. Los datos de identidad del operador autenticado son guardados dentro de variables de texto en el localStorage del navegador de forma insegura.

3.2. Capa de Servicio y Base de Datos Relacional

La API Express se aloja en el puerto local 4000. Al recibir una solicitud, esta cruza por middlewares encargados del formateo del cuerpo antes de invocar los enrutadores lógicos de control. Finalmente, los controladores interactúan de forma directa con el motor de base de datos local sqlite3 construyendo las instrucciones SQL mediante concatenación elemental de variables de entrada.

4. Hallazgos de Calidad de Software y Deuda Técnica

El diagnóstico estático indexó indicadores de alerta severos: 534 Code Smells activos, una complejidad ciclomática acumulada de 291 y una cobertura general de pruebas del cero por ciento.

4.1. CAL-001: Duplicación de Código Exacta en Cálculo de Mora

Ubicación: Archivos `frontend/src/utils/sonarDebt.js` y `backend/src/controllers/legacyD`
Severidad: Media

Descripción: Se identificaron funciones aritméticas idénticas orientadas al cálculo matemático de las tasas de mora legadas de deudores. Esta duplicación exacta infringe el principio de diseño de software DRY (Don't Repeat Yourself), incrementando las posibilidades de introducir regresiones asincrónicas durante tareas de mantenimiento en producción.

```
1 function calcularMoraLegacy(monto, dias) {  
2   let tasa = 0.05;  
3   if (dias > 30) tasa = 0.10;  
4   if (dias > 90) tasa = 0.25;  
5   return monto * tasa * (dias / 365);  
6 }
```

Listing 1: Código duplicado en controladores de deudas

4.2. CAL-002: Complejidad Ciclomática Excesiva en Evaluación de Riesgo

Ubicación: `backend/src/controllers/clientController.js` en la función `evaluateRiskCategory`.

Severidad: Alta

Descripción: La función presenta una complejidad lineal de valor 38 debida al uso excesivo de estructuras de variación de flujos condicionales condensadas en bloques

if-else profundamente anidados. Esto perjudica gravemente la comprensión lectora del código e impide estructurar suites de pruebas funcionales eficientes.

4.3. CAL-003: Manejo Desafortunado de Errores (Swallowed Exceptions)

Ubicación: Archivo backend/src/controllers/invoiceController.js

Severidad: Alta

Descripción: Se detectó que los bloques catch asociados al procesamiento de consultas de persistencia carecen por completo de instrucciones lógicas de respuesta, provocando que los fallos en runtime se silencien de manera crítica bloqueando la traza del error en auditorías operacionales.

```
1 try {  
2   const data = db.query(sql);  
3 } catch (err) {  
4   // Bloque vacio de control de fallos  
5 }
```

Listing 2: Bloque catch vacio silenciador de excepciones

4.4. CAL-004: Componentes Monolíticos Complejos (Funciones Diosas)

Ubicación: Archivo frontend/src/pages/Dashboard.jsx

Severidad: Alta

Descripción: El módulo concentra más de 800 líneas de código acoplando de manera indistinta lógica de dibujo de interfaces, peticiones de red por Axios, configuración de gráficas estadísticas y ordenamiento de arrays, rompiendo los principios de responsabilidad única de la arquitectura limpia.

4.5. CAL-005: Código Muerto y Variables Locales Obsoletas

Ubicación: Directorio de utilitarios del Frontend y helpers secundarios.

Severidad: Media

Descripción: Presencia masiva de declaraciones de variables que jamás son invocadas

y funciones legadas que permanecen inactivas tras cambios de diseño, expandiendo de manera artificial el peso del archivo bundle final enviado al usuario final.

4.6. CAL-006: Uso Sistémico e Inadecuado del Operador var

Ubicación: Totalidad de archivos controladores pertenecientes al Backend.

Severidad: Media

Descripción: La inicialización de punteros de memoria implementando el constructor antiguo var propicia el fenómeno de hoisting dentro del motor V8, alterando el ciclo de vida de los datos al dotarlos de un alcance global en lugar de un alcance de bloque delimitado.

4.7. CAL-007: Ausencia Total de Documentación y Comentarios Técnicos

Ubicación: Controladores matemáticos encargados del cálculo de amortizaciones de deudas.

Severidad: Media

Descripción: Los algoritmos financieros de alta densidad matemática no poseen anotaciones que clarifiquen el modelo abstracto subyacente, dificultando las actividades de transferencia tecnológica y onboarding de nuevos ingenieros al equipo de desarrollo.

4.8. CAL-008: Validación Mediante Cadenas de Texto Rígidas (Hardcoded Strings)

Ubicación: Módulo de validación de estados dentro de invoiceRoutes.js

Severidad: Media

Descripción: El flujo de control evalúa los estados operacionales de facturas contrastando strings planos literales directos en lugar de invocar enumeradores tipados estandarizados, favoreciendo fallos operativos debido a errores tipográficos manuales.

4.9. CAL-009: Complejidad Cognitiva Elevada en Event Loop

Ubicación: Rutas de filtrado de clientes en backend/src/controllers/clientController.js

Severidad: Alta

Descripción: La presencia de iteraciones anidadas de tipo for aplicadas sobre arreglos masivos de registros de clientes genera un bloqueo de ejecución síncrono sobre el hilo principal único de Node.js, degradando el rendimiento general de la API ante peticiones concurrentes.

4.10. CAL-010: Inexistencia Absoluta de Suites de Pruebas Automatizadas

Ubicación: Carpeta raíz del entorno del proyecto.

Severidad: Alta

Descripción: El entorno reporta un cero por ciento de cobertura debido a la ausencia total de marcos estandarizados de aserción automatizada (como Jest o Supertest), impidiendo validar la estabilidad lógica de la aplicación ante modificaciones del código fuente.

5. Hallazgos de Seguridad (OWASP Top 10)

5.1. SEC-001: Inyección SQL Estricta en Módulo de Autenticación

Clasificación OWASP: A03:2021 — Injection

Severidad: Crítica

Ubicación: Archivo backend/src/controllers/authController.js

Descripción Técnica: Las entradas provistas por los usuarios en los campos de correo y contraseña se concatenan directamente sin validación semántica previa dentro de la sentencia SQL enviada al motor sqlite3. Esto rompe el aislamiento de datos y permite alterar la consulta para evadir por completo las barreras lógicas del login.

```
1 const query = "SELECT * FROM users WHERE email = '"
2   + email + "' AND password = '" + password + "'";
3 db.query(query);
```

Listing 3: Sentencia SQL vulnerable por concatenacion directa

5.2. SEC-002: Ejecución Remota de Comandos (OS Command Injection)

Clasificación OWASP: A03:2021 — Injection

Severidad: Crítica

Ubicación: Archivo backend/src/controllers/adminController.js

Descripción: El endpoint administrativo expone el método nativo exec de Node.js alimentando la instrucción directamente con variables provenientes del query string de la URL. Un operador malicioso puede encadenar comandos del sistema operativo para tomar el control absoluto del entorno de alojamiento.

```
1 router.get('/lab/cmd', (req, res) => {
2   const cmd = req.query.cmd;
3   exec(cmd, (err, stdout, stderr) => {
4     res.json({ stdout });
5   });
6 });
```

Listing 4: Inyeccion de consola remota destructiva expuesta

5.3. SEC-003: Control de Acceso Roto mediante Token Predictible Artesanal

Clasificación OWASP: A01:2021 — Broken Access Control

Severidad: Crítica

Ubicación: Middleware backend/src/middleware/auth.js

Descripción: La verificación de privilegios delega su lógica en el parseo rudimentario de una cadena de caracteres separada por guiones intermedios que transporta explícitamente el rol del usuario. Al carecer de firma criptográfica verificable o cifrado simétrico, faculta a cualquier agente en la red a suplantar roles directivos modificando localmente el texto.

```
1 const weakAuth = (req, res, next) => {  
2   const token = req.headers['authorization'];  
3   const parts = token.split('-');  
4   req.user = { id: parts[1], role: parts[3] };  
5   next();  
6 };
```

Listing 5: Estructura de token artesanal debil

5.4. SEC-004: Fuga Masiva de Credenciales en Texto Plano

Clasificación OWASP: A04:2021 — Insecure Design

Severidad: Crítica

Ubicación: Ruta GET expuesta en /api/auth/users

Descripción: El enrutador publica de manera abierta a peticiones sin validar la tabla completa de registros de usuarios del sistema, exponiendo las contraseñas operativas almacenadas en texto claro legibles sin haber implementado de ninguna forma algoritmos de hashing seguros.

5.5. SEC-005: Exposición de Variables de Entorno del Servidor

Clasificación OWASP: A05:2021 — Security Misconfiguration

Severidad: Crítica

Ubicación: Endpoint de depuración administrado en /api/admin/debug

Descripción: El controlador responde enviando un volcado del objeto global process.env

del servidor de Node.js, filtrando en la red datos altamente sensibles como la semilla de simulación de llaves, rutas de carpetas de infraestructura y secretos de base de datos.

5.6. SEC-006: Configuración Permisiva de Cabeceras CORS

Clasificación OWASP: A05:2021 — Security Misconfiguration

Severidad: Alta

Descripción: La política de compartición de recursos de origen cruzado expone la directiva con el comodín asterisco, posibilitando que aplicaciones externas capturen información mediante scripts asíncronos cruzados desde el navegador.

5.7. SEC-007: Ausencia del Atributo HttpOnly en la Emisión de Cookies

Clasificación OWASP: A01:2021 — Broken Access Control

Severidad: Alta

Descripción: Los mecanismos de almacenamiento de cookies del cliente omiten configurar el atributo de seguridad HttpOnly, facilitando la extracción ilegítima de datos de sesión por parte de ataques de Cross-Site Scripting del lado del navegador.

5.8. SEC-008: Llave Criptográfica JWT Semilla Quemada en Código

Clasificación OWASP: A02:2021 — Cryptographic Failures

Severidad: Crítica

Ubicación: Módulo de inicialización en database.js

Descripción: La cadena de caracteres empleada como base criptográfica para verificar la integridad estructural de las identidades se encuentra expuesta en texto plano dentro de la solución, anulando por completo la confianza del ecosistema.

5.9. SEC-009: Inyección de Salto de Directorios (Path Traversal)

Clasificación OWASP: A01:2021 — Broken Access Control

Severidad: Alta

Ubicación: Endpoint de descarga de archivos en invoiceRoutes.js

Descripción: El parámetro encargado de ubicar los archivos adjuntos en el disco duro acepta de forma cruda caracteres de control lógicos de tipo punto-punto-barra, habilitando a operadores externos a descargar archivos de configuración del sistema operativo base.

5.10. SEC-010: Vulnerabilidad de Cross-Site Scripting Almacenado

Clasificación OWASP: A03:2021 — Injection

Severidad: Alta

Ubicación: Paneles de visualización de bitácoras de clientes morosos.

Descripción: Las entradas de texto de comentarios provistas por operadores del sistema se inyectan en el flujo de renderizado implementando la propiedad peligrosa dangerouslySetInnerHTML de React, propiciando la ejecución de scripts dañinos.

5.11. SEC-011: Ausencia de Mecanismos de Control de Tasa (Rate Limiting)

Clasificación OWASP: A05:2021 — Security Misconfiguration

Severidad: Alta

Descripción: La API carece de barreras limitadoras de peticiones por ventana temporal de tiempo, transformando al endpoint de logueo en un vector vulnerable al colapso por ataques masivos de fuerza bruta automatizada o denegación de servicios.

5.12. SEC-012: Puerta Trasera de Auto-login por Parámetro URL

Clasificación OWASP: A07:2021 — Identification Failures

Severidad: Crítica

Ubicación: Captura lógica en la raíz de enrutamiento del Frontend.

Descripción: El código intercepta las URL activas y, si localiza el string fijo estático `zap=auto`, salta los controles de autenticación tradicionales inyectando un token administrativo directamente en el almacenamiento de sesión local.

5.13. SEC-013: Redirecciones Abiertas Inseguras

Clasificación OWASP: A01:2021 — Broken Access Control

Severidad: Media

Descripción: Transferencia de flujos de navegación basados en parámetros externos sin validar contra listas blancas corporativas, facilitando ataques de phishing hacia los operadores de CarteraPro.

5.14. SEC-014: Ausencia de Cabeceras de Seguridad HSTS

Clasificación OWASP: A05:2021 — Security Misconfiguration

Severidad: Media

Descripción Técnica: El servidor web Express no implementa ni transmite en sus respuestas la cabecera Strict-Transport-Security (HSTS). Como consecuencia, la aplicación permite conexiones degradadas a través de canales no cifrados mediante el protocolo HTTP plano, lo que facilita ataques activos de interceptación de tráfico y secuestro de datos en tránsito por parte de intermediarios en la red local.

5.15. SEC-015: Fuga de Stack Traces ante Errores Internos HTTP 500

Clasificación OWASP: A05:2021 — Security Misconfiguration

Severidad: Media

Descripción Técnica: Al provocarse un fallo no controlado en la capa lógica del

backend, los enrutadores capturan el incidente pero devuelven al cliente una respuesta HTTP 500 que incluye un volcado crudo del stack trace del sistema operativo y las líneas del motor de ejecución. Esto revela de manera explícita la ruta de archivos físicos en el disco duro, nombres de funciones y dependencias, facilitando el diseño de exploits dirigidos de ingeniería inversa.

5.16. SEC-016: Manejo de Sesiones Infinitas sin Expiración Controlada

Clasificación OWASP: A07:2021 — Identification and Authentication Failures
Severidad: Media

Descripción Técnica: El backend carece de una política de revocación temporal de tokens. Una vez que se emite un identificador de sesión artesanal, este conserva su vigencia semántica de forma indefinida dentro de las validaciones de Express, extendiendo peligrosamente la ventana de compromiso en caso de que una estación de trabajo compartida sea abandonada por el operador financiero.

5.17. SEC-017: Falta de Validación de Esquemas y Tipos en Datos de Entrada

Clasificación OWASP: A03:2021 — Injection
Severidad: Alta

Descripción Técnica: Múltiples endpoints encargados de la actualización e inserción de balances y montos de facturación procesan las variables provenientes del cliente sin contrastarlas contra un esquema de validación rígido (como Joi o Yup). Esto propicia el procesamiento de caracteres especiales y desbordamientos lógicos, desestabilizando el comportamiento del motor de persistencia SQLite.

5.18. SEC-018: Exposición de Secretos de Desarrollo en el Bundle de Producción

Clasificación OWASP: A05:2021 — Security Misconfiguration
Severidad: Alta

Descripción Técnica: Durante el proceso de empaquetado y optimización con Vite,

las variables de depuración internas de la firma desarrolladora y URLs de APIs de prueba fueron inyectadas directamente en el código de producción visible por el cliente. Cualquier auditor o tercero puede extraer estas cadenas inspeccionando los archivos de script estáticos descargados en el navegador.

5.19. SEC-019: Ausencia Total de Cabeceras Content-Security-Policy (CSP)

Clasificación OWASP: A05:2021 — Security Misconfiguration

Severidad: Media

Descripción Técnica: Las respuestas HTTP del backend no incorporan políticas de seguridad de contenido mediante la cabecera CSP. Esto desprotege el entorno del cliente frente a inyecciones complejas, permitiendo que scripts de dominios maliciosos externos se carguen e inicien llamadas asíncronas no autorizadas simulando interacciones legítimas del usuario.

5.20. SEC-020: Implementación de Algoritmia Hashing MD5 Obsoleta

Clasificación OWASP: A02:2021 — Cryptographic Failures

Severidad: Media

Descripción Técnica: El sistema emplea la función criptográfica rota MD5 para generar los identificadores únicos y nombres indexados de las facturas cargadas en la carpeta de subidas. Al ser un algoritmo vulnerable a colisiones, un atacante puede predecir o sobrescribir documentos de otros deudores alterando los flujos lógicos de auditoría.

5.21. SEC-021: Divulgación Tecnológica Explícita en Cabecera X-Powered-By

Clasificación OWASP: A05:2021 — Security Misconfiguration

Severidad: Baja

Descripción Técnica: El servidor Express transmite por defecto en todas las respuestas la cabecera HTTP X-Powered-By con el valor explícito de Node.js. Esta filtración trivial

simplifica las tareas de reconocimiento automatizado por parte de atacantes, quienes pueden mapear de forma ágil la infraestructura y buscar vulnerabilidades específicas asociadas al motor.

5.22. SEC-022: Falta de Cabeceras Contra Suplantación de Tipos MIME

Clasificación OWASP: A05:2021 — Security Misconfiguration

Severidad: Media

Descripción Técnica: La aplicación no implementa la directiva de seguridad X-Content-Type-Options con el valor nosniff. Como consecuencia, navegadores web antiguos o desactualizados pueden ignorar el tipo MIME declarado e interpretar archivos de texto o imágenes descargadas como scripts ejecutables, propiciando vectores indirectos de XSS.

5.23. SEC-023: Vulnerabilidad ante Secuestro de Clics (Click-jacking)

Clasificación OWASP: A01:2021 — Broken Access Control

Severidad: Media

Descripción Técnica: La aplicación web omite la cabecera X-Frame-Options o las directivas frame-ancestors en sus respuestas. Esto faculta que la plataforma CarteraPro sea embebida dentro de marcos e iframe pertenecientes a portales hostiles de terceros, permitiendo capturar de forma invisible los clics e interacciones de los analistas legítimos del sistema.

5.24. SEC-024: Inyección de Encabezados HTTP (HTTP Response Splitting)

Clasificación OWASP: A03:2021 — Injection

Severidad: Alta

Descripción Técnica: Determinados endpoints toman parámetros directamente del query string de la URL y los vuelcan de manera cruda dentro de las funciones de establecimiento de cabeceras de respuesta. Al no sanitizar saltos de línea (CRLF), un atacante puede inyectar encabezados arbitrarios o manipular la estructura HTTP

interpretada por el navegador.

5.25. SEC-025: Almacenamiento Inseguro de Sesiones en Local-Storage

Clasificación OWASP: A01:2021 — Broken Access Control

Severidad: Alta

Descripción Técnica: La persistencia de la identidad del analista y sus tokens opacos se realiza exclusivamente dentro del localStorage del navegador. Debido a que este almacenamiento carece de mecanismos de aislamiento y protección frente a código JavaScript local, cualquier vulnerabilidad de tipo XSS confiere acceso de lectura inmediato y compromiso total de la sesión activa.

5.26. SEC-026: Falta de Sanitización Estricta en la Subida de Archivos

Clasificación OWASP: A01:2021 — Broken Access Control

Severidad: Alta

Descripción Técnica: El módulo de carga de facturas valida de manera laxa la extensión de los archivos adjuntos cruzando cadenas de caracteres, pero prescinde de inspeccionar la firma real de bytes (magic numbers) en el servidor. Esto permite a un atacante enmascarar scripts maliciosos ejecutables asignándoles extensiones inocuas para introducirlos en el sistema de almacenamiento local.

5.27. SEC-027: Endpoint de Depuración Reset Público Activo en Producción

Clasificación OWASP: A01:2021 — Broken Access Control

Severidad: Crítica

Ubicación: Ruta expuesta en /api/test/reset

Descripción Técnica: Se localizó un enrutador activo destinado a vaciar de manera integral e inmediata todas las tablas de deudores, clientes e historiales de facturas de la base SQLite para agilizar las pruebas de laboratorio de la firma desarrolladora. Al no contar con validación de tokens ni middlewares de autorización, cualquier usuario

de la red puede detonar esta llamada provocando la pérdida total de los activos de información.

5.28. SEC-028: Claves de Cifrado Simétrico Estáticas Compartidas

Clasificación OWASP: A02:2021 — Cryptographic Failures

Severidad: Alta

Descripción Técnica: Los registros sensibles de la base de datos sqlite3 que requieren protección en reposo se almacenan aplicando un cifrado simétrico elemental. No obstante, la lógica utiliza una única clave estática hardcodeada compartida para la totalidad de las filas. El compromiso o descubrimiento de esta semilla por accesos no autorizados expone instantáneamente los datos históricos de la organización.

6. Análisis de Composición de Software (SCA)

El análisis automatizado arrojó un total de 25 vulnerabilidades activas por obsolescencia tecnológica perimetral. A continuación, se desglosa el catálogo extendido individual de las dependencias analizadas:

6.1. DEP-001: Biblioteca request Deprecada y Vulnerable (CVE-2023-28155)

Severidad: Crítica

Descripción Técnica: La aplicación utiliza la librería request 2.88.2, la cual se encuentra oficialmente obsoleta y sin mantenimiento desde el año 2020. Expone al sistema a ataques de Server-Side Request Forgery (SSRF), permitiendo a atacantes remotos usar el servidor como proxy para escanear e interactuar de forma maliciosa con recursos internos de la red local institucional.

6.2. DEP-002: Paquete Transitivo form-data Inseguro por Semillas Previsibles

Severidad: Crítica

Descripción Técnica: El módulo secundario encargado del manejo de flujos multipart emplea generadores matemáticos pseudoaleatorios previsibles nativos del motor de JavaScript para estructurar los delimitadores de carga. Esto facilita que un tercero anticipe e indexe de forma maliciosa las rutas de subida de archivos adjuntos financieros.

6.3. DEP-003: Biblioteca Axios Vulnerable a la Fuga de Tokens CSRF

Severidad: Alta

Descripción Técnica: La versión de Axios instalada en la paquetería del cliente presenta un fallo de diseño lógico al transferir de manera involuntaria las cabeceras de autorización y credenciales privadas hacia servidores ajenos controlados por terceros durante eventos de redirección automática.

6.4. DEP-004: Express Framework Vulnerable a Ejecuciones XSS en Redirección

Severidad: Alta

Descripción Técnica: La compilación Express 4.17.1 expone fallos donde comandos de secuencias de comandos maliciosos provistos por el usuario se ejecutan de forma reflejada cuando el método nativo `res.redirect` resuelve variables de navegación introducidas directamente por el cliente sin aplicar filtros de escape estructurados.

6.5. DEP-005: JsonWebToken Obsoleto con Omisión de Firma (Algoritmo None)

Severidad: Alta

Descripción Técnica: La biblioteca instalada para gestionar la validación simétrica es susceptible a elusiones criptográficas debido a fallos lógicos heredados que aceptan el parámetro `none` como algoritmo válido, permitiendo forjar identidades administrativas válidas removiendo la firma criptográfica del encabezado.

6.6. DEP-006: Lodash Desactualizado Propenso a Prototype Pollution

Severidad: Alta

Descripción Técnica: Exposición a inyecciones lógicas de control mediante la manipulación recursiva de objetos utilitarios ejecutando métodos nativos de asignación profunda de la librería, lo que faculta a alterar el comportamiento interno de variables globales en la memoria del servidor de Node.js.

6.7. DEP-007: Moment.js Legacy con Fallos de Expresiones Regulares (ReDoS)

Severidad: Media

Descripción Técnica: La paquetería Moment.js desactualizada del entorno procesa cadenas de texto de fechas complejas aplicando motores RegExp ineficientes. Un atacante

puede estructurar entradas de fechas maliciosas que desbordan el tiempo de procesamiento, congelando el hilo único asíncrono y provocando la denegación de servicios inmediata.

6.8. DEP-008: Multer Vulnerable a Agotamiento de Recursos RAM

Severidad: Alta

Descripción Técnica: La versión de Multer empleada para interceptar flujos multipart carece de controles rígidos sobre la cantidad de campos y tamaños máximos en el búfer. Ráfagas masivas de solicitudes pueden agotar la memoria asignada al proceso, provocando caídas abruptas de la API del backend.

6.9. DEP-009: Body-Parser Bloqueante por Cargas Masivas de Datos

Severidad: Media

Descripción Técnica: El módulo encargado de transformar los cuerpos de solicitudes URL-encoded anidadas exhibe deficiencias operativas ante estructuras masivas de datos, consumiendo excesivos ciclos de procesamiento y degradando el tiempo de respuesta general de los endpoints del sistema.

6.10. DEP-010: Driver SQLite3 Bindings Vulnerable a Fallos de Segmentación

Severidad: Alta

Descripción Técnica: Las capas compiladas nativas de C++ del driver sqlite3 presentan inestabilidades lógicas cuando se enlazan parámetros nulos o anómalos directamente en consultas nativas, arrojando excepciones críticas que interrumpen abruptamente la ejecución de la API de Express.

6.11. DEP-011: Servidor de Desarrollo Vite Vulnerable a Path Traversal

Severidad: Alta

Descripción Técnica: El motor local integrado de Vite empleado para gestionar el entorno del cliente presenta brechas que permiten eludir las restricciones de carpetas locales del host, permitiendo extraer archivos estáticos situados fuera del directorio raíz asignado al proyecto.

6.12. DEP-012: Nodemon Inseguro con Inyecciones en Scripts de Consola

Severidad: Media

Descripción Técnica: La herramienta secundaria de recarga en caliente de desarrollo arrastra dependencias transitivas obsoletas que facilitan la denegación de servicios local si se configuran scripts de análisis manipulados de forma maliciosa.

6.13. DEP-013: Dotenv Obsoleto ante Caracteres de Escape Inválidos

Severidad: Baja

Descripción Técnica: La versión instalada de dotenv presenta fallos lógicos menores al procesar y parsear archivos de entorno que incorporan caracteres especiales mal configurados, provocando truncamientos imprevistos en las cadenas de conexión de base de datos leídas.

6.14. DEP-014: Cors Package Legacy con Asignación de Orígenes Laxos

Severidad: Media

Descripción Técnica: La librería encargada de las directivas de origen cruzado arrastra un diseño por defecto permisivo si el programador omite declarar de forma explícita el arreglo estricto de dominios autorizados, heredando las directivas abiertas analizadas en SEC-006.

6.15. DEP-015: Multer File Type Spoofing en Detección de Extensiones

Severidad: Alta

Descripción Técnica: Los complementos de detección de extensiones asociados a la subida de archivos se confían enteramente en los datos legibles provistos en las cabeceras HTTP de la petición, facilitando el engaño y enmascaramiento de archivos peligrosos tal como se documentó en SEC-026.

7. Valoración de Riesgos e Impacto de Negocio

Para determinar la prioridad de intervención de los hallazgos identificados se aplicó la fórmula paramétrica: $\text{Riesgo} = \text{Probabilidad (1-3)} \times \text{Impacto (1-3)}$. El catálogo reporta un total de 54 vulnerabilidades consolidadas (14 Críticas, 29 Altas, 1 Media, 10 Bajas).

Tabla 3: Matriz Multidimensional de Riesgos Financieros y Técnicos

ID	Hallazgo Clave	P	I	Score	Consecuencia Comercial
SEC-001	Inyección SQL Autenticación	3	3	Crítico (9)	Bypass total de login; acceso no autorizado a carteras de crédito.
SEC-002	Inyección SQL Búsqueda	3	3	Crítico (9)	Exfiltración masiva de credenciales y contraseñas de analistas.
SEC-003	Token Predictible WeakAuth	3	3	Crítico (9)	Suplantación de identidades directivas mediante forjado de tokens.
SEC-004	Inyección de Comandos RCE	3	3	Crítico (9)	Parálisis operativa por toma de control absoluta del servidor web.
SEC-005	Ejecución Remota vía eval()	3	3	Crítico (9)	Ejecución de scripts arbitrarios en runtime; exfiltración de memoria.
SEC-007	Credenciales Hardcodeadas	3	3	Crítico (9)	Compromiso irreversible de llaves AWS cloud y certificados RSA.
SEC-008	Fuga masiva process.env	3	3	Crítico (9)	Exposición de llaves maestras institucionales.
SEC-012	Backdoor Parámetro URL	3	3	Crítico (9)	Acceso trivial a analíticas financieras mediante ?zap=auto.
SEC-013	Contraseñas en Texto Plano	3	3	Crítico (9)	Acceso directo a datos financieros sin requerir descifrado o cracking.

Continúa en la siguiente página

Tabla 3 – continuación

ID	Hallazgo	Clave	P	I	Score	Consecuencia Comercial
DEP-001	Librería Request De-	precada	3	3	Crítico (9)	Exposición permanente de la API a vectores SSRF de la red local.
CAL-008	Concatenación SQL	sistémica	3	3	Crítico (9)	Deuda técnica que replica la inyección en cada endpoint nuevo.
SEC-009	SSRF Fetch Adminis-	trativo	3	2	Alto (6)	Uso del servidor como pivote para escanear infraestructura interna.
SEC-010	XSS Almacenado en	Clientes	3	2	Alto (6)	Secuestro de sesiones activas de analistas que consulten notas.
SEC-011	Path Traversal (LFI)		3	2	Alto (6)	Lectura de archivos de configuración restringidos del sistema operativo.
CAL-001	Código Masivo	Duplicado	3	2	Alto (6)	Correcciones de parches ineficaces por dispersión de funciones.
CAL-003	Ausencia de Tests	Unitarios	3	2	Alto (6)	Imposibilidad de validar regresiones de software en el pipeline.

8. Correlación y Validación Cruzada de Hallazgos

Para certificar la veracidad científica de las incidencias documentadas en este informe y descartar de forma absoluta la presencia de falsos positivos introducidos por los escáneres automáticos, el equipo auditor ejecutó una validación cruzada tripartita. Se contrastaron las detecciones estáticas (SAST por SonarQube) contra las alertas de runtime (DAST por OWASP ZAP), requiriendo de manera obligatoria una confirmación por explotación o inspección de código manual para validar la existencia material del hallazgo.

A continuación, se detalla la matriz de correlación cruzada de las principales

infracciones:

Tabla 4: Correlación Híbrida de Detección de Infracciones Técnicas

Hallazgo	Rev. Manual	SonarQube	OWASP ZAP	Verificación
SEC-001 (Inyección SQL)	Sí	Sí	Sí	Confirmado Real
SEC-002 (Inyección Comandos RCE)	Sí	Sí	Sí	Confirmado Real
SEC-003 (Mecanismo de Tokens)	Sí	No	No	Confirmado Real
SEC-004 (Fuga de Credenciales)	Sí	No	Sí	Confirmado Real
SEC-005 (Fuga de process.env)	Sí	Sí	Sí	Confirmado Real
SEC-012 (Backdoor Auto-login)	Sí	No	No	Confirmado Real
CAL-003 (Excepciones Silenciosas)	Sí	Sí	No	Confirmado Real
DEP-001 (Biblioteca Request SSRF)	Sí	Sí	No	Confirmado Real

9. Anexo de Evidencias Técnicas (Registro de Laboratorio)

En cumplimiento estricto con los requerimientos establecidos en las pautas de evaluación, a continuación se adjuntan los registros y cargas útiles (*payloads*) recolectados de forma directa en el entorno de pruebas de software controlado:

9.1. Evidencia 1: Reproducción Manual de Bypass de Autenticación (SEC-001)

La consulta del login concatena variables directamente. Se reproduce el bypass enviando caracteres de escape mediante una petición POST estructurada:

```
1 curl -X POST http://localhost:4000/api/auth/login \  
2 -H "Content-Type: application/json" \  
3 -d '{"username": "'\'' OR '\''1'\''=\'\''1'\'' --",  
4     "password": "x"}'
```

Listing 6: Petición curl para bypass de autenticación

La consulta resultante ejecutada en el motor SQLite3 destruye el aislamiento lógico de la contraseña:

```
1 SELECT * FROM users  
2 WHERE username = '' OR '1'='1' --'
```

Listing 7: Sentencia SQL interceptada en runtime

9.2. Evidencia 2: Exfiltración de Tabla de Usuarios vía SQLi UNION (SEC-002)

El parámetro *q* del endpoint de búsqueda de clientes permite inyectar sentencias transaccionales complejas para extraer la tabla de credenciales en texto plano:


```

1 curl "http://localhost:4000/api/clients/search?\
2 q=%25' %20UNION %20SELECT %20id,username,\
3 password,role,email,fullName %20FROM %20users %20--"

```

Listing 8: Inyeccion UNION para extraer tabla de usuarios

9.3. Evidencia 3: Ejecución Remota de Comandos (RCE via exec - SEC-004)

El endpoint administrativo permite interactuar directamente con el sistema operativo host debido a la desprotección del método `child_process.exec`:

```

1 curl "http://localhost:4000/api/admin/lab/cmd\
2 ?cmd=id%3Buname%20-a"

```

Listing 9: Activacion del endpoint RCE

Respuesta en formato JSON devuelta de forma abierta por la API:

```

1 {
2   "command": "id;uname -a",
3   "stdout": "uid=1000(node) gid=1000(node)
4             Linux risk-lab-host..."
5 }

```

Listing 10: Respuesta del servidor con informacion del sistema

9.4. Evidencia 4: Exposición de Secretos en Local File Inclusion (SEC-013)

Invocación utilizando secuencias de salto de directorio relativo (`../`) para exfiltrar las llaves privadas y credenciales del entorno de despliegue:

```

1 curl "http://localhost:4000/api/admin/lab/\
2 file?path=../../.env"

```

Listing 11: Path traversal para acceder al archivo `.env`

9.5. Evidencia 5: Activación de la Backdoor de Auto-login por URL (SEC-022)

El componente frontend inyecta una sesión administrativa completa al interceptar los parámetros de automatización del proxy DAST:

```
1 http://localhost:5173/clients?zap=auto
```

Listing 12: URL de activacion de la backdoor

Esto fuerza la escritura automática en el almacenamiento persistente del navegador:

```
1 localStorage.setItem('token', 'token-1-admin-admin');
```

Listing 13: Estructura inyectada en el localStorage

10. Conclusiones y Plan de Intervención Priorizado

La plataforma CarteraPro Risk Lab presenta deficiencias lógicas e inestabilidades estructurales severas tanto en su diseño arquitectónico como en sus capas operacionales de control de acceso, arrojando un dictamen general desfavorable desde la perspectiva de la ingeniería de software y la ciberseguridad industrial. El sistema exhibe un estado latente de vulneración inminente, por lo que se dictamina con carácter obligatorio suspender de forma inmediata cualquier exposición externa en la red y ejecutar el plan de remediación priorizado estructurado en tres horizontes temporales rígidos:

10.1. Horizonte 1: Acciones de Emergencia Inmediatas (1 a 2 semanas)

- **Remediación SQLi:** Reemplazar todas las sentencias concatenadas de los controladores financieros migrando a consultas parametrizadas mediante sentencias preparadas nativas de sqlite3.
- **Aislamiento de Consola RCE:** Remover de las rutas de Express los enrutadores de depuración expuestos que invocan las funciones peligrosas `exec` o `eval` del sistema operativo `host`.
- **Remoción de Backdoors:** Eliminar del frontend las capturas lógicas de parámetros URL estáticos que inyectan sesiones falsas en el `localStorage` eludiendo el login.

10.2. Horizonte 2: Mitigaciones y Parches a Corto Plazo (1 mes)

- **Cifrado Adaptativo de Credenciales:** Eliminar el guardado de variables en texto plano e incorporar funciones criptográficas de hashing lento implementando

la biblioteca bcrypt con factor de trabajo robusto.

- **Adopción de Estándares de Identidad:** Migrar el parseo predecible de strings hacia JSON Web Tokens (JWT) firmados criptográficamente mediante claves asimétricas robustas guardadas de manera externa.
- **Saneamiento SCA de Empaquetado:** Forzar una depuración masiva de dependencias mediante limpiezas npm y migrar el obsoleto cliente request hacia axios actualizado para remover los vectores CVE activos.

10.3. Horizonte 3: Sostenibilidad e Ingeniería a Mediano Plazo (3 meses)

- **Refactorización de Complejidad:** Romper las funciones monolíticas complejas anidadas mapeadas por SonarQube e introducir el patrón de diseño Estrategia para controlar la complejidad ciclomática del software.
- **Cultura de Testing Automatizado:** Diseñar e instanciar suites de pruebas unitarias y funcionales asíncronas con Jest y Supertest para revertir de manera definitiva el indicador de cobertura del 0.0% y asegurar la estabilidad de la solución.

10.4. Recomendaciones Complementarias de Ingeniería

Se aconseja capacitar de forma periódica al equipo de desarrollo en prácticas de codificación segura bajo el estándar OWASP Top 10, implementar escaneos automatizados SAST/SCA integrados directamente en las canalizaciones de CI/CD para bloquear despliegues con Quality Gates en rojo, y estructurar un programa formal de divulgación responsable de vulnerabilidades (Bug Bounty).

10.5. Declaración Final del Equipo Auditor

El grupo de auditoría técnica e ingeniería inversa certifica que ha actuado con estricta diligencia profesional, documentando hallazgos reales, explotables y verificables mediante evidencias físicas dentro de este entorno controlado de laboratorio. Las brechas expuestas representan un riesgo material alto para los activos financieros de la organización, por lo que se insta a tratar la Fase 1 de este plan como una acción de emergencia inmediata.